

Section 2

Fibonacci Heaps



Fibonacci Heaps

Properties:

- Binomial heap that can have more than one tree of any degree.
- We can remove at most one child from each node. (We mark nodes that have had a child removed)

Very loose structure; could just be a list of all root nodes. We use amortized analysis to get the runtimes we want.



Formalizing Amortized Analysis

- We use amortized analysis when the work done by slow operations can be attributed evenly to a predictable number of fast operations.
- With the *Potential Method*, we “bank” time in a potential function, ϕ , on the fast operations, then “cash out” on the slow operations.
- Formally, we define $\Delta\phi_o$ as the change in potential after operation o . Then the amortized time is calculated as

$$T_{amortized}(o) = T_{actual}(o) + C \cdot \Delta\phi_o$$

for a constant C that disappears in big O .

- When we “cash out”, the amortized time is smaller than the actual time, and ϕ decreases (usually to 0).



Amortizing a Fibonacci Heap

- We define ϕ as

$$\phi = t + 2m$$

for a Fibonacci heap with t trees and m marked nodes.

- This looks really arbitrary, but we will soon see why it works.

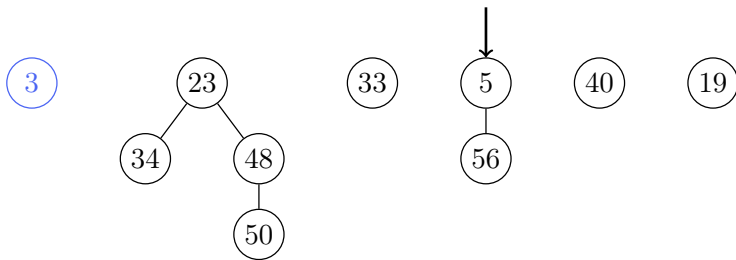


Fibonacci Heap INSERT

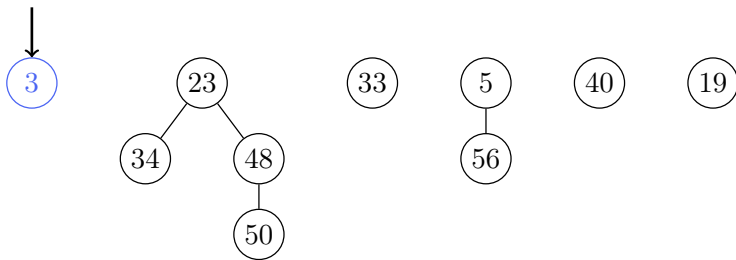
- To insert, we simply add a new tree to the heap with the node we are adding as the root.
- We maintain a pointer to the minimum element, so if this is the new minimum, update the pointer.
- $O(1)$.



Fibonacci Heap INSERT Example

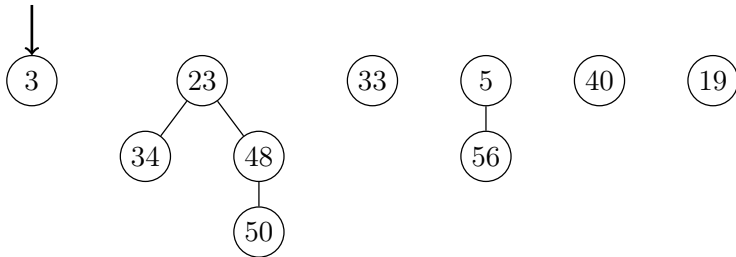


Fibonacci Heap INSERT Example



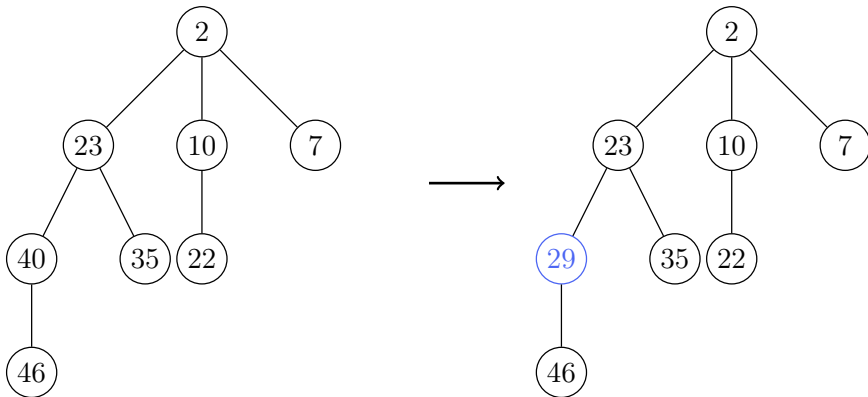
Fibonacci Heap FINDMIN

Because we maintain a pointer to the minimum element, accessing it is $O(1)$.



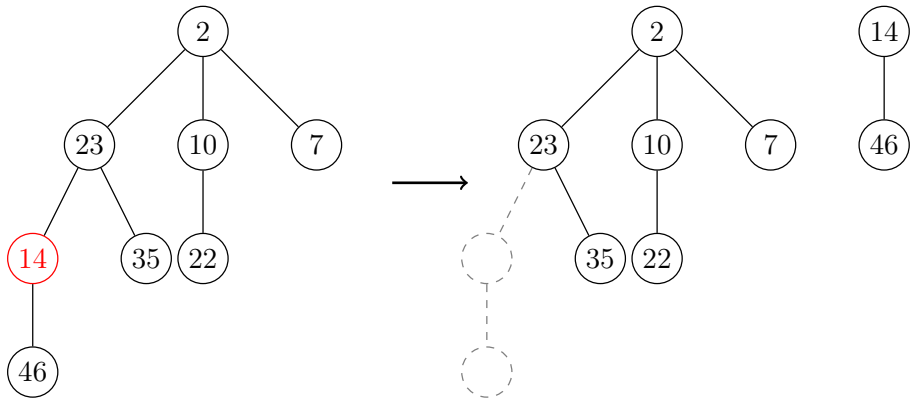
Fibonacci Heap DECREASEKEY

1. Decrease the key. If the new value maintains the heap property, we are done.



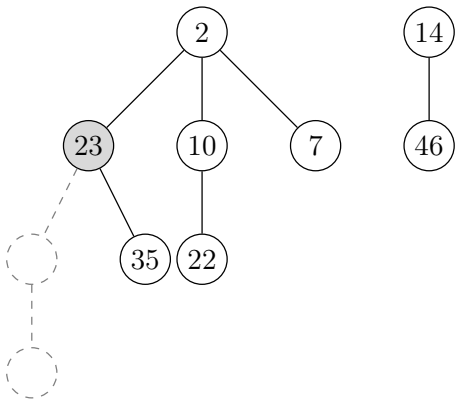
Fibonacci Heap DECREASEKEY

2. Otherwise, cut the decreased key (and its subtree) out into a new tree.



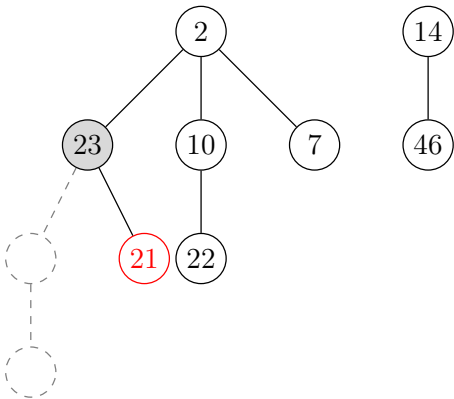
Fibonacci Heap DECREASEKEY

3. If the parent was unmarked, mark it. Otherwise, cut it out, unmark it, and repeat for this node's parent.



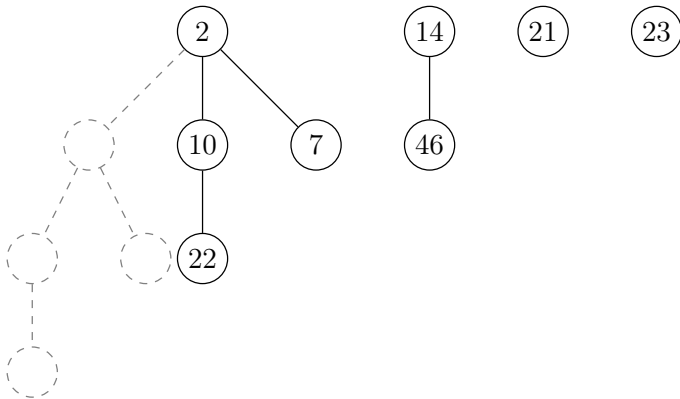
Fibonacci Heap DECREASEKEY

Now, if we cut out 35, we would also cut out 23:



Fibonacci Heap DECREASEKEY

Now, if we cut out 35, we would also cut out 23:



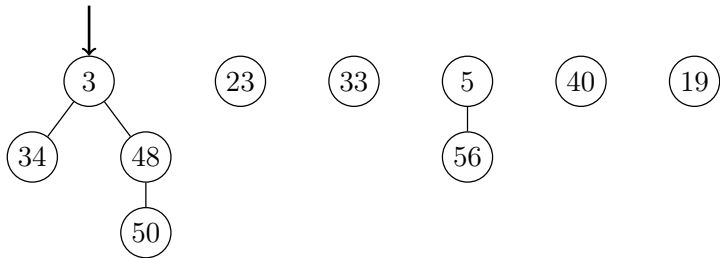
Fibonacci Heap DECREASEKEY Analysis

- Suppose we made k cuts in total. Then we added k trees in $O(k)$.
- Of all the nodes we cut out, only the first was unmarked, but all are unmarked now. Including the final extra node we marked, we have that marked nodes changed by $-(k-1) + 1 = -k + 2$.
- Remember $\phi = t + 2m$ for t trees and m marked nodes, and $T_{amortized}(o) = T_{actual}(o) + C \cdot \Delta\phi$.
- Thus, $\Delta\phi = k + 2(-k + 2) = -k + 4$.
- So our amortized time is $O(1)$.
- Notice our choice of ϕ was important here.



Fibonacci Heap DELETEMIN Phase 1

Cut out the minimum node (which must be a root) and make its children roots of new trees.



Fibonacci Heap DELETEMIN Phase 1

Cut out the minimum node (which must be a root) and make its children roots of new trees.



With d children, this operation is $O(d)$. We will see later that $O(d) = O(\log n)$ for a fibonacci heap with n nodes.



Fibonacci Heap DELETEMIN Phase 2

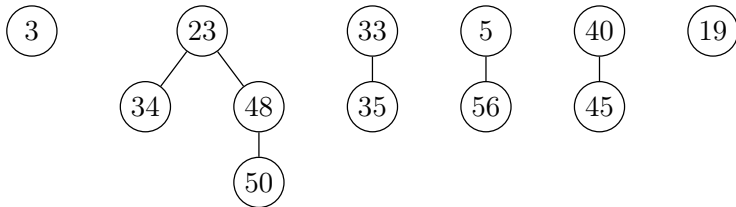
- Now we need to update the minimum pointer.
- Until now, we haven't enforced much structure, so in general, there might now be n roots, and finding the minimum is $O(n)$.
- However, if we “clean up,” future calls to DELETEMIN could be faster, and could give us a $\Delta\phi$ that helps in amortized analysis.
- Like the last slide, I will assume the largest degree of any root is $O(\log n)$, and prove this later.



Fibonacci Heap DELETMIN Phase 2

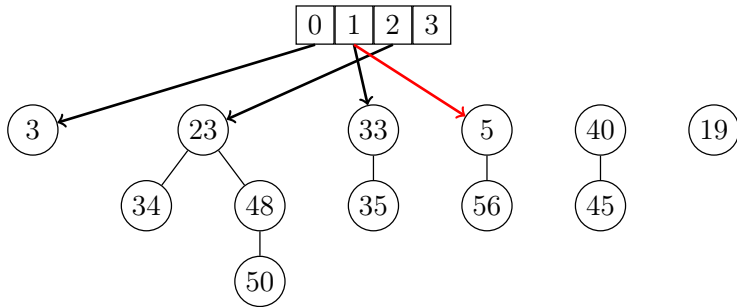
1. Create an array of length $O(\log n)$ to track trees of every possible degree.

0	1	2	3
---	---	---	---



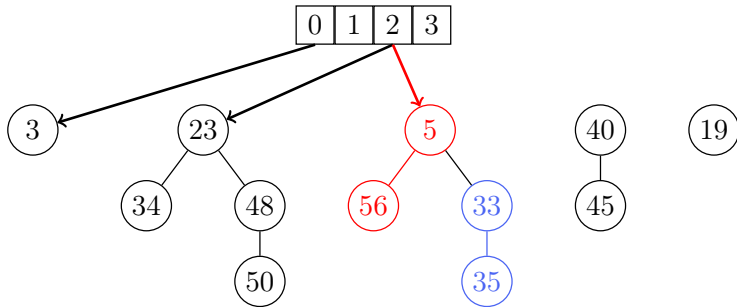
Fibonacci Heap DELETMIN Phase 2

2. Put each tree into an index of the array. If there is already a tree there, merge them to create a tree of degree +1.



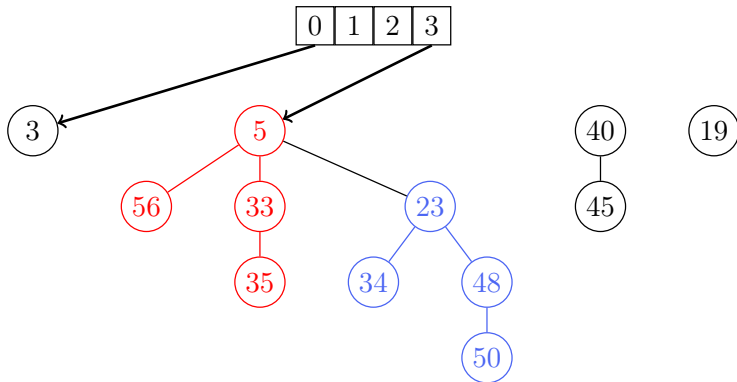
Fibonacci Heap DELETEmIN Phase 2

2. Put each tree into an index of the array. If there is already a tree there, merge them to create a tree of degree +1.



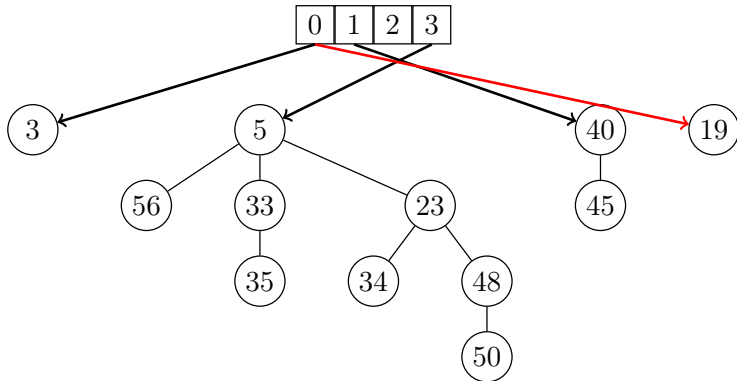
Fibonacci Heap DELETEmIN Phase 2

2. Put each tree into an index of the array. If there is already a tree there, merge them to create a tree of degree +1.



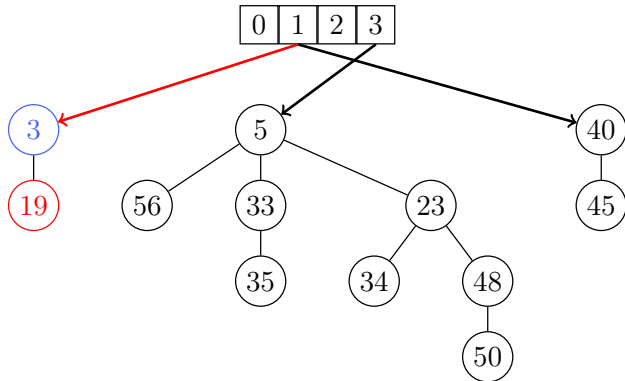
Fibonacci Heap DELETMIN Phase 2

2. Put each tree into an index of the array. If there is already a tree there, merge them to create a tree of degree +1.



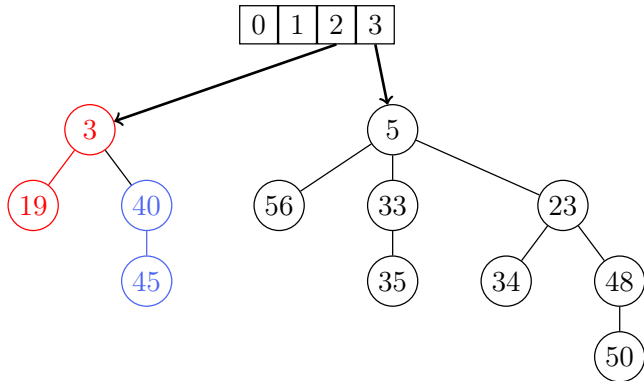
Fibonacci Heap DELETMIN Phase 2

2. Put each tree into an index of the array. If there is already a tree there, merge them to create a tree of degree +1.



Fibonacci Heap DELETMIN Phase 2

2. Put each tree into an index of the array. If there is already a tree there, merge them to create a tree of degree +1.



Fibonacci Heap DELETEMIN Phase 2 Analysis

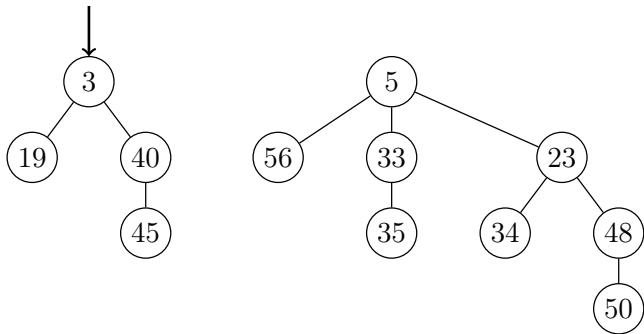
- We need to allocate the array and visit each of m starting roots for $O(\log n + m)$ actual runtime.
- $\Delta\phi = \Delta(\#trees) = O(\log n) - m$. (final - initial)
- Thus, the amortized runtime for this phase is

$$\begin{aligned} & O(\log n + m) + C \cdot (O(\log n) - m) \\ &= O(\log n) + O(m) + C \cdot O(\log n) - C \cdot m \\ &= O(\log n) \quad \text{(with large enough } C\text{)} \end{aligned}$$



Fibonacci Heap DELETEMIN Phase 3

Find the minimum of the $O(\log n)$ roots that remain in $O(\log n)$ time.



Fibonacci Heap DELETEmIN Recap

Of all 3 phases, the slowest runtime was $O(\log n)$, so the overall runtime is $O(\log n)$.



Questions?



Fibonacci Heap Degree Bound Proof

- Earlier, we assumed that the largest degree of any root is $O(\log n)$. How do we know? This is where Fibonacci comes in.
- We focus on one tree with degree d and n nodes total. We want

$$d \leq O(\log n)$$

$$d \leq \log_a n \quad (\text{for some } a > 1)$$

$$d \cdot \log a \leq \log n \quad (\text{if } a \leq 1, \text{ the } \leq \text{ would flip})$$

$$\log a^d \leq \log n$$

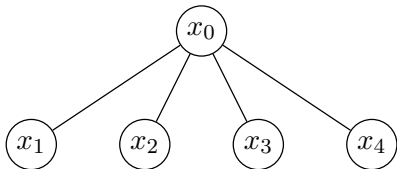
$$a^d \leq n$$

- Thus, we need to show that n grows exponentially with respect to d .



Fibonacci Heap Degree Bound Proof

- To do this, let's look for a lower bound on n for a given d .
- Consider a tree of degree 4:

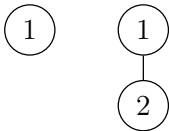


- When x_3 was added, it must have been degree 2.
- Because each node can have at most one child removed, x_3 must now have degree at least 1. Similarly, x_4 must have degree at least 2.
- In general, the i^{th} child must have degree at least $i - 2$.



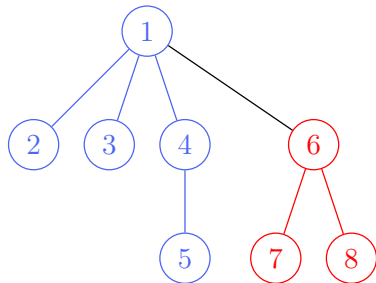
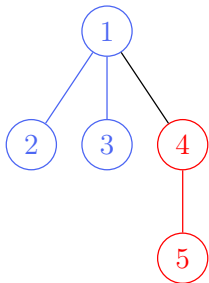
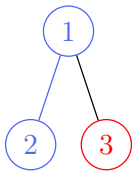
Fibonacci Heap Degree Bound Proof

- We can use this to build up the minimum trees for each degree.
- Minimum for degrees 0 and 1 (base cases) are trivial:



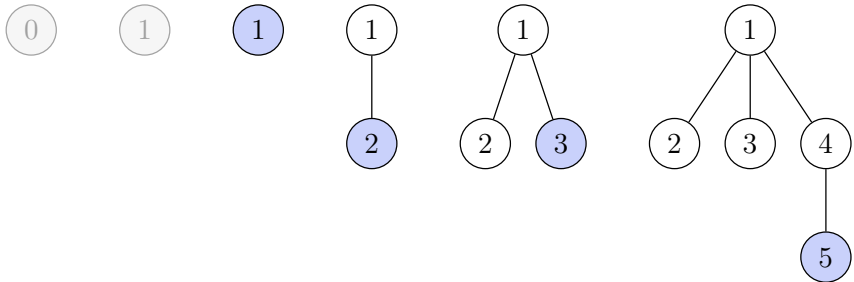
Fibonacci Heap Degree Bound Proof

- Then for tree of degree d we can add tree of degree $d - 2$ as a child of tree $d - 1$.
- Degrees 2–4 are as follows:



Fibonacci Heap Degree Bound Proof

- Fibonacci!



- We know that the growth of Fibonacci approaches $\Phi \approx 1.618$, so $n \geq \Phi^d$. which satisfies our goal.

